STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: JAVA_001_OOPS
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: OOPS Concepts using Java with AI	STATUS: In Progress

Practice Worksheet – OOPS Concepts using Java with AI

Master core Object-Oriented Programming principles: Encapsulation in billing models, Multilevel Inheritance hierarchies, Polymorphism via Abstract Classes, and Custom Business Exceptions.

KEY LEARNING OBJECTIVES:

- Implement encapsulated domain models with defensive copying and input validation.
- Design multilevel inheritance hierarchies with method overriding and `super` chaining.
- Formulate polymorphic business calculation systems using Abstract Classes.
- Develop custom checked/unchecked exception hierarchies for retail domain validation.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Model clean Java classes adhering to SOLID principles and proper access modifiers.
Step 2: Implement constructor overloading, getter/setter data constraints, and `toString()` methods.
Step 3: Override base methods with `@Override` and ensure proper polymorphic dispatch.
Step 4: Write sample execution logic in the workspace showing instantiation and output.

EXERCISE 4.1: Q001 & Q004: ENCAPSULATION & TIERED DISCOUNT BILL GENERATION

Problem Statement: Create an encapsulated `FoodBill` class with private fields: `billId`, `customerName`, `totalAmount`, and `discountRate`. Enforce validation: `totalAmount` must be > 0. Implement a method `calculateFinalAmount()` that applies 10% discount if total > $500, 20% discount if total > $1000, and returns the net payable amount.

Reference Baseline / Code / Scenario:

```
FoodBill bill = new FoodBill(101, "Amandeep", 1200.0); double netPayable = bill.calculateFinalAmount();
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Class Attributes & Private Encapsulation: _____

[2] Validation & Discount Logic: _____

[3] Constructor & Accessor Methods: _____

EXERCISE 4.2: Q015: MULTILEVEL INHERITANCE – CREDIT CARD REWARD SYSTEM

Problem Statement: Design a 3-tier multilevel inheritance structure: Base class `Account` (accountNumber, balance), derived class `CardAccount` (cardNumber, creditLimit), and final class `RewardCreditCard` (rewardPoints, cashbackRate). Override `processPayment(double amount)` to deduct balance, calculate 2% cashback reward points, and call `super.processPayment()`.

Reference Baseline / Code / Scenario:

```
class RewardCreditCard extends CardAccount { @Override public void processPayment(double amt) { ... } }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Inheritance Hierarchy Class Diagram: _____

[2] Method Overriding with super(): _____

[3] Reward Computation Logic: _____

EXERCISE 4.3: Q010: ABSTRACT CLASSES & POLYMORPHIC BONUS CALCULATION

Problem Statement: Create an abstract class `Employee` with fields `id`, `name`, `baseSalary` and an abstract method `double calculateBonus()`. Implement two concrete subclasses: `Manager` (bonus = 25% of base salary + performance allowance) and `Developer` (bonus = 15% of base salary + project incentive). Demonstrate polymorphism using an `Employee` array.

Reference Baseline / Code / Scenario:

```
Employee[] staff = { new Manager("Alice", 80000), new Developer("Bob", 60000) };
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Abstract Class Definition: _____

[2] Subclass Implementations: _____

[3] Polymorphic Array Dispatch: _____

EXERCISE 4.4: Q018 & Q020: CUSTOM EXCEPTION & CONTACT VALIDATION ENGINE

Problem Statement: Create a custom checked exception `InvalidProductPriceException` and an unchecked `InvalidContactException`. In a `ProductService` class, implement `registerProduct(String sku, double price, String phone)` that throws `InvalidProductPriceException` if price <= 0 and `InvalidContactException` if phone does not contain exactly 10 digits.

Reference Baseline / Code / Scenario:

```
if (price <= 0) throw new InvalidProductPriceException("Product price must be positive");
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Custom Exception Class Definitions: _____

[2] Validation Method with throw/throws: _____

[3] Try-Catch-Finally Handling Block: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Encapsulation Integrity:** All fields private; access mediated by validated getters/setters.
- [ ] **Inheritance Design:** Multilevel hierarchy demonstrates true 'is-a' relationships.
- [ ] **Polymorphism Accuracy:** Abstract methods overridden cleanly; runtime dynamic dispatch proven.
- [ ] **Exception Architecture:** Custom exceptions inherit correctly from Exception or RuntimeException.
- [ ] **Code Cleanliness:** Follows standard Java naming conventions and clean code structure.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is composition often preferred over inheritance in modern software architecture?*
- *How does using custom domain exceptions improve system maintainability compared to throwing generic RuntimeExceptions?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) **[ ] NEEDS REVISION (Resubmit Corrected Work)** **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Best Practices